IDEATION PHASE

Brainstorm & Idea Prioritization Template

Date: 03 October 2026
Team ID: 6aba117e538da4003d63184c

Project Name: Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks: 4 Marks

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Team Members

Name	Role
Pranaya S	Team Lead
Deepika P	Member
Sajitha K	Member
Thenmozhi R	Member
Sharmila M	Member

Problem Statement

In a ServiceNow application, users may have different levels of access to records based on their roles and the values stored in specific fields. A suitable access control mechanism is required to prevent unauthorized users from viewing restricted records.

The selected problem is to **create a script-controlled Access Control List (ACL) that restricts record access based on the value of the Branch field**.

For this project, records are maintained in the u_institution_details table. The Branch field contains values such as **ECE, EEE and CSE**. The requirement is that users with the appropriate role should be able to access the permitted records, while unauthorized users should be denied access. Administrators should retain full access.

Selected Objective

To implement a script-controlled ACL in ServiceNow that controls record-level access based on the Branch field and user roles.

Step-2: Brainstorm, Idea Listing and Grouping

The team discussed different approaches for controlling access to the records.

Brainstormed Ideas

No.	Idea	Description	Group
1	Role-Based Access	Use ServiceNow roles to control access to records.	Access Control
2	Field-Based Access	Restrict records depending on the value of the Branch field.	Record Security
3	Script-Controlled ACL	Use an ACL script to evaluate the user's role and allow or deny access.	Access Control
4	READ ACL	Control whether a user can view records.	CRUD Security
5	CREATE ACL	Control whether a user can create new records.	CRUD Security
6	WRITE ACL	Control whether a user can modify existing records.	CRUD Security
7	DELETE ACL	Control whether a user can delete records.	CRUD Security
8	Administrator Override	Allow administrators to retain full access to the records.	Exception Handling
9	User Testing	Test access using users having different roles.	Validation
10	Unauthorized Access Verification	Verify that users without required permissions cannot access restricted records.	Security Testing

Grouping of Ideas

Group 1 – User and Role Management

- Create users
- Create roles
- Assign appropriate roles to users

Group 2 – Table and Data Management

- Create u_institution_details
- Add the required fields

- Create records with different Branch values

Group 3 – Access Control

- READ ACL
- CREATE ACL
- WRITE ACL
- DELETE ACL

Group 4 – Script-Based Security

- Check whether the user has the required role
- Allow administrator access
- Deny unauthorized users

Group 5 – Testing and Verification

- Test with authorized users
- Test with unauthorized users
- Test with administrator
- Verify record-level access

Step-3: Idea Prioritization

The team prioritized the ideas according to their importance to the project.

Priority	Idea	Reason for Priority
1	Script-Controlled ACL	Core requirement for controlling record access
2	Field-Based Access	Determines which records should be accessible
3	Role-Based Access	Identifies users who are permitted to access records
4	READ ACL	Prevents unauthorized users from viewing restricted records
5	CREATE ACL	Controls creation of new records
6	WRITE ACL	Controls modification of records
7	DELETE ACL	Controls deletion of records

Priority	Idea	Reason for Priority
8	Administrator Override	Ensures administrators retain full access
9	User and Role Creation	Provides users for ACL testing
10	Access Verification	Confirms that the implemented security rules work correctly

Final Prioritized Solution

After brainstorming and prioritization, the team selected the following implementation approach:

1. Create the required users and roles.
2. Create the u_institution_details table.
3. Add records with different Branch values.
4. Configure a **READ ACL** with the required role and Branch condition.
5. Use a script to evaluate the user's permissions.
6. Configure **CREATE, WRITE and DELETE ACLs** using the corresponding roles.
7. Test the ACLs using users with different combinations of roles.
8. Verify that unauthorized users cannot access restricted records.
9. Verify that administrators retain full access.

Selected Final Idea

Script-Controlled ACL for Field-Based Record Access

The final idea combines **role-based access control, field-value-based record restriction and script-based permission validation** to provide controlled access to ServiceNow records.